LeetCoach New Device Handoff

Generated: September 12, 2026

Purpose

LeetCoach is a Chrome extension that helps users learn how to think through LeetCode problems instead of immediately showing final answers.

Core philosophy:

Don’t give me the answer. Teach me how to discover it.

The project is currently an MVP full-stack learning tool:

- Chrome extension side panel built with React, TypeScript, Vite, and Manifest V3.
- FastAPI backend using an in-memory mock dataset.
- No SQLite, PostgreSQL, accounts, authentication, or real LLM API yet.
- Current AI-like behavior is mocked and deterministic.

Repository Status Before Device Switch

- Current branch: ‘master‘.
- ‘master‘ is even with ‘origin/master‘ at the time this handoff was generated.
- The only visible untracked item was ‘pet-work/‘, which appears unrelated to the LeetCoach app.
- Main handoff file inside the repo: ‘roadmap.md‘.

If the new device does not have the project yet, clone it from GitHub:

```
git clone https://github.com/<your-github-username>/LeetCoach.git
cd LeetCoach
```

If ‘pet-work/‘ matters, copy it manually from the old device. It is not part of the tracked LeetCoach codebase.

How To Run On The New Device

Frontend / Extension

```
npm install
npm run build
```

The Chrome-loadable extension bundle is generated in:

```
dist/
```

Load it in Chrome:

1. Open ‘chrome://extensions‘.
2. Enable Developer Mode.
3. Click ‘Load unpacked‘.
4. Select the repo’s ‘dist/‘ folder.
5. Open a supported LeetCode problem page such as ‘https://leetcode.com/problems/two-sum/‘.
6. Open the LeetCoach side panel.

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

The backend runs at:

```
http://localhost:8000
```

Quick health check:

```
curl http://localhost:8000/health
```

Expected response:

```
{"status":"ok"}
```

Current Architecture

Frontend:

- React
- TypeScript
- Vite
- Chrome Extension Manifest V3
- Chrome Side Panel API
- Active tab detection for ‘https://leetcode.com/problems/*‘

Backend:

- FastAPI
- In-memory mock problem dataset
- CORS configured for local extension development
- Canonical approach metadata with stable approach IDs
- Hint service abstraction
- Solution service abstraction

Database:

- Not implemented yet.
- Planned local MVP database: SQLite.
- Future migration plan: SQLAlchemy + Alembic + PostgreSQL.

AI:

- No real LLM provider yet.
- Mock/static guidance only.
- Future providers may include OpenAI, Gemini, or OpenRouter.

Current User Flow

1. User opens a LeetCode problem page.
2. Extension side panel detects the active tab URL.
3. URL parser extracts the problem slug from ‘leetcode.com/problems/{slug}‘.
4. Side panel fetches problem analysis from the local FastAPI backend.
5. Backend returns canonical approaches for the detected problem.
6. UI shows:
   - Problem title
   - Hints section
   - Approach selector
   - Selected approach details
   - Generate Answer section
   - Thinking process toggle

The extension currently supports only ‘leetcode.com‘, not ‘leetcode.cn‘.

Seeded Problems

The backend mock dataset currently includes:

- ‘two-sum‘
- ‘valid-parentheses‘
- ‘longest-substring-without-repeating-characters‘

Unknown problems return:

```
{
  "detail": "This problem is not available in the local LeetCoach dataset
yet."
}
```

Backend API Surface

Health:

```
GET /health
```

Problem analysis:

```
GET /api/problems/{slug}
```

Primary approach hints:

```
GET /api/problems/{slug}/hints
```

Approach-specific hints:

```
GET /api/problems/{slug}/approaches/{approach_id}/hints
```

Approach-specific mock solution:

```
POST /api/problems/{slug}/approaches/{approach_id}/solution
```

Request body:

```
{
  "language": "python"
}
```

Supported languages:

- ‘python‘
- ‘java‘
- ‘javascript‘
- ‘cpp‘

Important product rule:

Generated answers, hints, thinking process, and future starter code must always be tied to a stable ‘approach.id‘.

The backend must never generate generic problem-level solutions.

Canonical Approach Rules

Each problem owns a stable list of approaches.

Each approach has:

- ‘id‘
- ‘name‘
- ‘pattern‘
- ‘summary‘
- ‘isPrimary‘
- ‘displayOrder‘
- ‘thinkingSteps‘
- ‘hints‘

Validation currently rejects mock data when:

- A problem has no primary approach.
- A problem has more than one primary approach.
- A problem has duplicate approach IDs.

Exactly one approach per problem is primary.

Before a user selects an approach, hints use the primary approach.

After a user selects an approach, hints and generated answers use the selected ‘approach.id‘.

Frontend File Map

Core app:

- ‘src/App.tsx‘: coordinates active problem, selected approach, thinking expansion, hints, and generated answer composition.
- ‘src/main.tsx‘: React entry point.
- ‘src/styles.css‘: all side panel styling.

Active problem detection:

- ‘src/problemUrl.ts‘: LeetCode problem URL parsing and title normalization.
- ‘src/useActiveProblem.ts‘: reads active Chrome tab and updates on tab changes/navigation.

Problem analysis:

- ‘src/api/problemAnalysis.ts‘: problem-analysis API client and response normalization.
- ‘src/useProblemAnalysis.ts‘: problem-analysis request state.

Hints:

- ‘src/api/approachHints.ts‘: hint API client.
- ‘src/hooks/useApproachHints.ts‘: lazy hint loading, caching, progressive reveal, stale response protection.
- ‘src/components/HintPanel.tsx‘: hint UI.

Generated answers:

- ‘src/api/approachSolution.ts‘: generated-answer API client.
- ‘src/hooks/useApproachSolution.ts‘: generated-answer request state.
- ‘src/components/GenerateAnswer.tsx‘: language selector and answer display.

Approaches and thinking:

- ‘src/components/ApproachSelector.tsx‘: numbered approach buttons.
- ‘src/components/ApproachDetails.tsx‘: selected approach name, pattern, summary, close action.
- ‘src/components/ThinkingProcess.tsx‘: thinking-process toggle and steps.

Other components:

- ‘src/components/ProblemHeader.tsx‘
- ‘src/components/AnalysisStatus.tsx‘

Chrome extension:

- ‘public/manifest.json‘
- ‘src/background.ts‘
- ‘sidePanel.html‘

Backend:

- ‘backend/app/main.py‘: FastAPI app, mock dataset, routes.
- ‘backend/app/services/hint_service.py‘: hint lookup from canonical approach.
- ‘backend/app/services/solution_service.py‘: approach-specific mock solution generation.
- ‘backend/requirements.txt‘

Current UI Behavior

When no supported LeetCode problem is open:

- Shows LeetCoach MVP intro copy.
- Shows an ‘Open LeetCode‘ button.

When a supported problem is open:

- Intro copy is hidden to save sidebar space.
- Problem title is shown prominently.
- Hints are shown above approaches.
- No hint is revealed automatically.
- Approach buttons are labeled ‘Approach 1‘, ‘Approach 2‘, etc.
- Approach names and patterns remain hidden until selection.
- Clicking selected approach again collapses it.
- Selected approach details can also be closed with a ‘Close‘ action.
- Generate Answer appears only after selecting an approach.
- Thinking process remains collapsed until explicitly opened.

Recent UI polish:

- Hints and Approaches are visually distinct.
- Hint cards have wider reading width, more padding, and more line height.
- Reveal animations are quick and subtle.
- ‘prefers-reduced-motion‘ is respected.

Verification Checklist

Frontend:

```
npm run build
```

Backend:

```
cd backend
python -m compileall app
```

```
uvicorn app.main:app --reload
```

Endpoint checks:

```
curl http://localhost:8000/health
curl http://localhost:8000/api/problems/two-sum
curl http://localhost:8000/api/problems/two-sum/hints
curl http://localhost:8000/api/problems/two-sum/approaches/brute-force/hints
```

Example generated-answer test:

```
curl -X POST http://localhost:8000/api/problems/two-sum/approaches/hash-
map/solution \
  -H "Content-Type: application/json" \
  -d "{\"language\":\"python\"}"
```

Manual Chrome checks:

- Load ‘dist/‘ as unpacked extension.
- Open ‘https://leetcode.com/problems/two-sum/‘.
- Confirm title appears.
- Confirm hints are not fetched until ‘Get first hint‘.
- Confirm ‘Show next hint‘ reveals one hint at a time.
- Confirm approach names are hidden before selection.
- Confirm selected approach details appear after clicking an approach.
- Confirm clicking the same approach collapses selected details.
- Confirm Generate Answer appears only after approach selection.
- Confirm switching approach clears generated answer and collapses thinking.
- Confirm unsupported problems show the local dataset fallback.
- Stop backend and confirm retryable connection error state.

Known Limitations

- No SQLite yet.
- No SQLAlchemy models yet.
- No Alembic migrations yet.
- No PostgreSQL yet.
- No user accounts or auth.
- No real LLM provider.
- No starter-code generator yet.
- No related questions yet.
- No follow-up questions yet.
- Backend README is slightly behind the current API surface; ‘roadmap.md‘ is the better source of truth.
- The extension assumes the backend is running at ‘http://localhost:8000‘.

Recommended Next Milestone

Add focused automated tests before adding the database layer.

Suggested tests:

- URL parsing and title normalization.
- Problem-analysis response normalization.
- Hint endpoint behavior.
- Solution endpoint behavior.
- Backend dataset validation.
- Hook behavior for lazy hints and generated answers.

After tests:

1. Add SQLite.
2. Add SQLAlchemy models for problems, approaches, hints, related problems, starter templates.
3. Add Alembic migrations.
4. Seed 10-20 curated LeetCode problems.
5. Preserve the existing ‘approach.id‘ contract.

Project Management Rule

Keep updating ‘roadmap.md‘ after meaningful changes.

This file is used as a ChatGPT/Codex PM handoff file so future sessions can quickly recover context.

Minor UI-only tweaks can be summarized briefly.

Most Important Design Constraint

Approaches are canonical.

Hints, thinking steps, generated answers, and future starter code must always come from the selected or primary canonical approach.

Never infer approach identity from:

- Button index
- Pattern name
- Display text
- Array position

Use only the stable backend-provided ‘approach.id‘.